

Lucas Augusto da Silva Cardoso (202200332300)

DCA3703 | Programação Paralela | Relato de experimento

TAREFA 12: AVALIAÇÃO DE ESCALABILIDADE

1. EXPERIMENTO

Avalie a escalabilidade do seu código de Navier-Stokes utilizando algum nó de computação do NPAD. Procure identificar gargalos de escalabilidade e reporte o seu progresso em versões sucessivas da evolução do código otimizado. Comente sobre a escalabilidade, a escalabilidade fraca e a escalabilidade fortes das versões.

2. CONCEITOS RELEVANTES

- **Desempenho, speedup e eficiência:** desempenho representa a rapidez da execução. O speedup mede o ganho em relação à execução com uma thread, S(p)=T(1)/T(p). A eficiência indica o aproveitamento relativo das threads, (E(p)=S(p)/p.
- **Escalabilidade:** capacidade de manter a eficiência ao aumentar os recursos e o problema. A forte mantém o problema fixo; a fraca mantém a eficiência mesmo quando o problema cresce menos que os recursos.
- **Gargalos e sobrecarga:** trechos sequenciais limitam o ganho paralelo, conforme a Lei de Amdahl. O gerenciamento das threads e as sincronizações também consomem tempo, especialmente quando há pouco trabalho por thread.
- **Localidade de memória e SIMD:** o processamento por linhas favorece acessos contíguos à memória e o uso da cache. A diretiva “omp simd” orienta a vetorização do laço interno.

3. MÁQUINA UTILIZADA

Os experimentos foram executados no nó “r2n50” do “NPAD/UFRN”, equipado com dois processadores “AMD EPYC 7713”, com 64 núcleos físicos cada, totalizando 128 núcleos físicos e 256 threads de hardware, distribuídos em oito domínios NUMA.

Para a avaliação, foram utilizadas “1, 4, 8, 16 e 32 threads”, com uma thread por núcleo físico. A afinidade foi configurada com “OMP_PLACES=cores” e “OMP_PROC_BIND=spread”. Os códigos foram compilados com “GCC 8.5.0 e OpenMP”, e os resultados foram coletados com o “PaScal Analyzer”.

4. CÓDIGO

Foram desenvolvidas três versões para avaliar a evolução do desempenho e da escalabilidade.

4.1 VERSÃO 1 (paralelização com collapse(2))

Distribui as iterações dos dois laços da malha entre as threads. A cada passo, utiliza três regiões paralelas: uma para atualizar os pontos internos e duas para copiar os contornos.

Figura 1: Versão 1

```
#include "navierStokes.h"

void evoluir(Grade *g, int passos) {
   int n=g->n; double inv=(double)(n-1)*(n-1), fator=0.2/inv;
   double *u=g->u,*v=g->v,*un=g->un,*vn=g->vn;

   for (int p=0;p<passos;p++) {
      #pragma omp parallel for collapse(2) schedule(static)
      for (int i=1;i<n-1;i++) {
         for (int j=1;j<n-1;j++) {
            ATUALIZAR;
         }
      }
      /* Duas regioes de contorno como na tarefa11, sem corrida nos cantos. */
      #pragma omp parallel for schedule(static)
```

```
    for (int i=1;i<n-1;i++) {
      un[(size_t)i*n]=u[(size_t)i*n]; vn[(size_t)i*n]=v[(size_t)i*n];
      un[(size_t)i*n+n-1]=u[(size_t)i*n+n-1]; vn[(size_t)i*n+n-1]=v[(size_t)i*n+n-1];
    }
    #pragma omp parallel for schedule(static)
    for (int j=0;j<n;j++) {
      un[j]=u[j];vn[j]=v[j];
      size_t k=(size_t)(n-1)*n+j;un[k]=u[k];vn[k]=v[k];
    }
    double *t=u;u=un;un=t;t=v;v=vn;vn=t;
  }
  g->u=u;g->v=v;g->un=un;g->vn=vn;
}
```

4.2 VERSÃO 2 (distribuição por linhas e SIMD)

Distribui as linhas entre as threads e aplica omp simd ao laço interno. Os contornos são inicializados nos dois buffers e permanecem constantes, eliminando suas cópias a cada passo.

A mudança reduz o trabalho repetido e passa de três regiões paralelas para uma por passo.

Figura 2: Versão 2

```
#include "navierStokes.h"

void evoluir(Grade *g, int passos) {
  int n=g->n; double inv=(double)(n-1)*(n-1), fator=0.2/inv;
  double *u=g->u,*v=g->v,*un=g->un,*vn=g->vn;
  for (int p=0;p<passos;p++) {
    #pragma omp parallel for schedule(static)
    for (int i=1;i<n-1;i++) {
      #pragma omp simd
      for (int j=1;j<n-1;j++) {
        ATUALIZAR;
      }
    }
    double *t=u;u=un;un=t;t=v;v=vn;vn=t;
  }
  g->u=u;g->v=v;g->un=un;g->vn=vn;
}
```

4.2 VERSÃO 3 (região paralela persistente)

Mantém uma única região paralela durante todos os passos. Cada thread recebe cópias privadas dos ponteiros e realiza localmente a troca dos buffers.

A barreira implícita do “omp for” garante que todas as atualizações terminem antes do próximo passo. Os ponteiros privados dispensam uma região “single” e uma barreira adicional para a troca. Essa versão também utiliza inicialização paralela da malha.

Figura 3: Versão 3

```
#include "navierStokes.h"

void evoluir(Grade *g, int passos) {
  int n=g->n; double inv=(double)(n-1)*(n-1), fator=0.2/inv;
  double *u=g->u,*v=g->v,*un=g->un,*vn=g->vn;
  #pragma omp parallel firstprivate(u,v,un,vn)
  {
    for (int p=0;p<passos;p++) {
      #pragma omp for schedule(static)
      for (int i=1;i<n-1;i++) {
        #pragma omp simd
        for (int j=1;j<n-1;j++) {
          ATUALIZAR;
        }
      }
      /* Barreira do omp for termina todas as escritas antes da troca.
        Ponteiros privados dispensam single e segunda barreira. */
      double *t=u;u=un;un=t;t=v;v=vn;vn=t;
    }
  }
  if (passos%2) { double *t=u;u=un;un=t;t=v;v=vn;vn=t; }
  g->u=u;g->v=v;g->un=un;g->vn=vn;
}
```

5. RESULTADOS

Foram avaliadas três versões com 1, 4, 8, 16 e 32 threads. Cada configuração foi executada cinco vezes, com 300 passos de simulação. As entradas i1, i2, i3 e i4 correspondem, respectivamente, a malhas internas de 256 × 256, 512 × 512, 1.024 × 1.024 e 2.048 × 2.048 pontos.

A análise considera o programa completo, identificado como região 0 no PaScal.

5.1. TEMPO DE EXECUÇÃO

Tabela 1: Tempo de execução

Threads	V1 (ms)	V2 (ms)	V3 (ms)
1	3.985,34	2.413,94	2.474,23
4	1.151,75	751,89	622,29
8	673,68	472,37	321,84
16	437,33	337,83	172,42
32	344,99	300,97	102,48

A V2 reduz o tempo em relação à V1 em todas as configurações apresentadas. A V3 apresenta o menor tempo nas execuções paralelas, chegando a ser 3,37 vezes mais rápida que a V1 e 2,94 vezes mais rápida que a V2, com 32 threads.

5.1. EFICIÊNCIA

Figura 4: Eficiência da versão 1

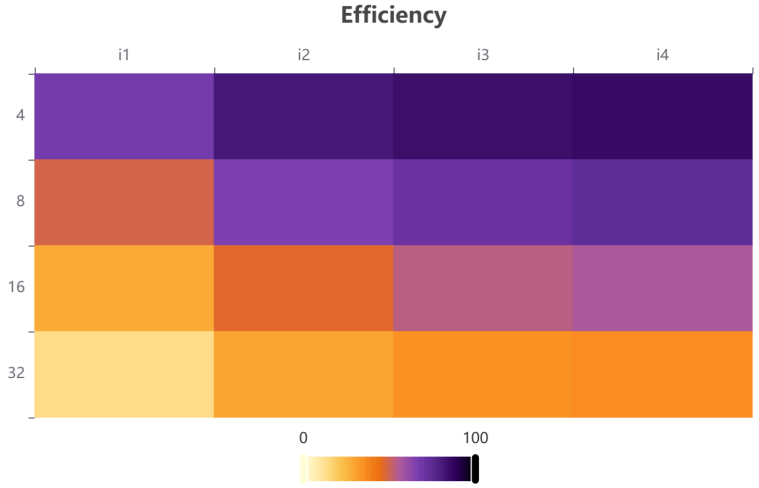


Figura 5: Eficiência da versão 2

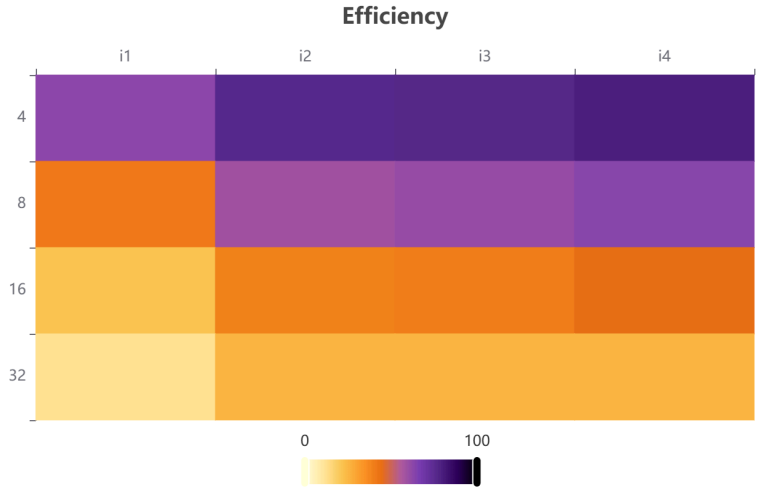
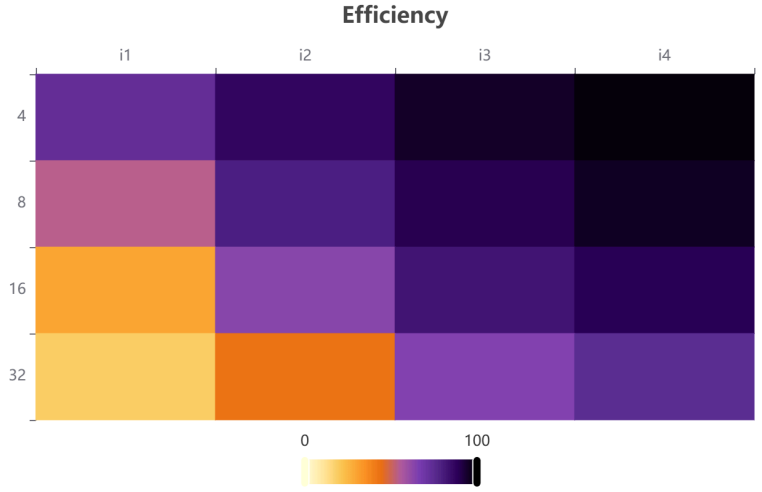


Figura 5: Eficiência da versão 3



A V3 atingiu a maior eficiência nas maiores entradas. A redução do trabalho serial e da sobrecarga permite que uma parcela maior do tempo seja dedicada ao processamento paralelo da malha.

Na menor entrada, sua eficiência cai, assim como nas demais versões. Nesse caso, há pouco trabalho por thread para compensar os custos de sincronização e gerenciamento.

5.1. ESCALABILIDADE GERAL

Figura 5: Escalabilidade geral da versão 1

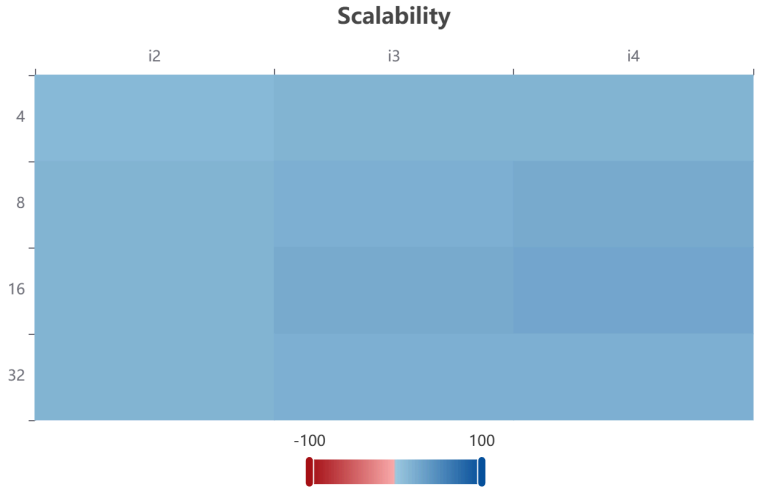


Figura 6: Escalabilidade geral versão 2

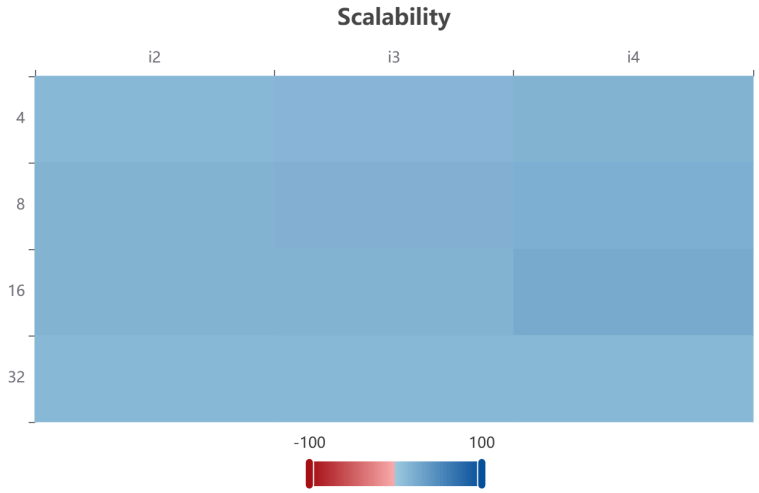
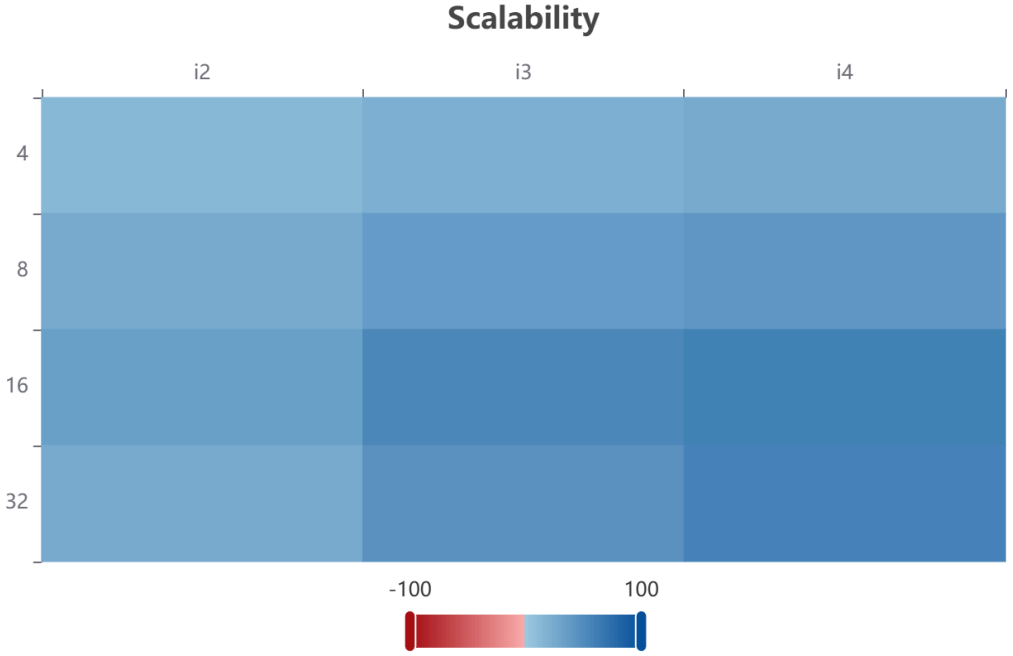


Figura 7: Escalabilidade geral versão 3



A V3 foi a que melhor aproveitou o crescimento do problema. Com 32 threads, sua eficiência aumentou de 12,4% em i1 para 75,4% em i4.

5.1. ESCALABILIDADE FORTE

Figura 8: Escalabilidade forte da versão 1

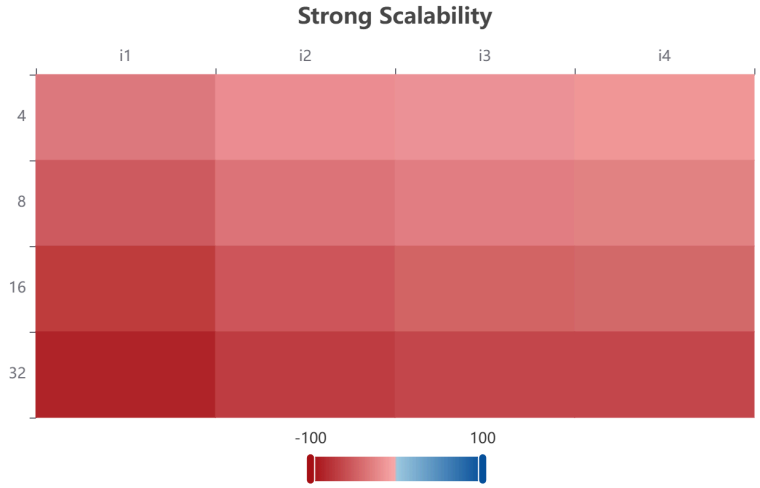


Figura 9: Escalabilidade forte versão 2

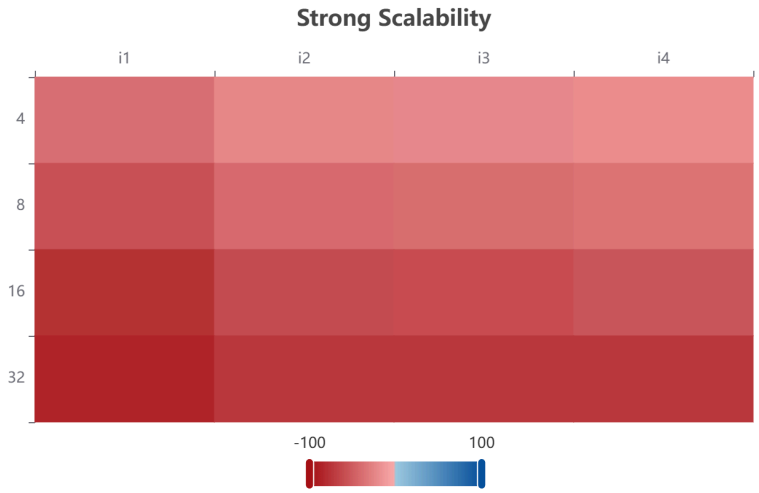
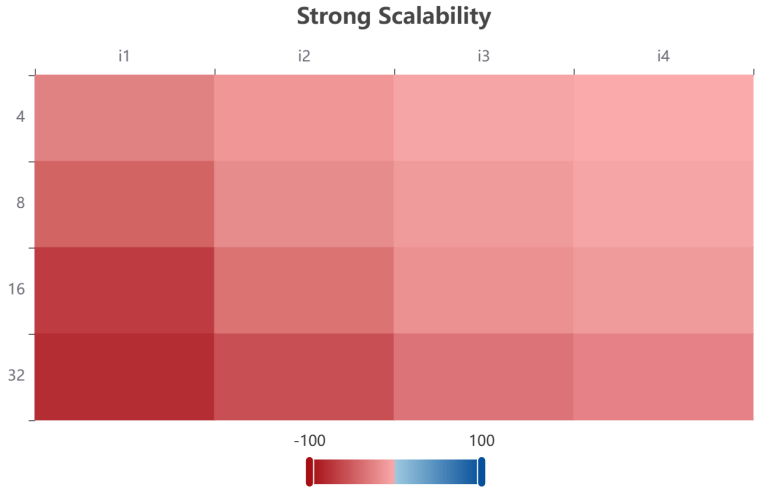


Figura 10: Escalabilidade forte versão 3



A V3 apresentou o melhor comportamento nas maiores malhas ao aumentar as threads com a entrada fixa. Em i4, passar de 16 para 32 threads produziu uma aceleração de 1,68×, contra 1,27× na V1 e 1,12× na V2.

Sua vantagem é coerente com a redução da fração serial e da sobrecarga repetida. Ainda assim, a barreira entre passos continua necessária, e seu custo ganha importância conforme o trabalho é dividido entre mais threads. Isso ajuda a explicar por que a V3 também perde desempenho com 32 threads na menor entrada.

5.1. ESCALABILIDADE FRACA

Figura 11: Escalabilidade fraca da versão 1

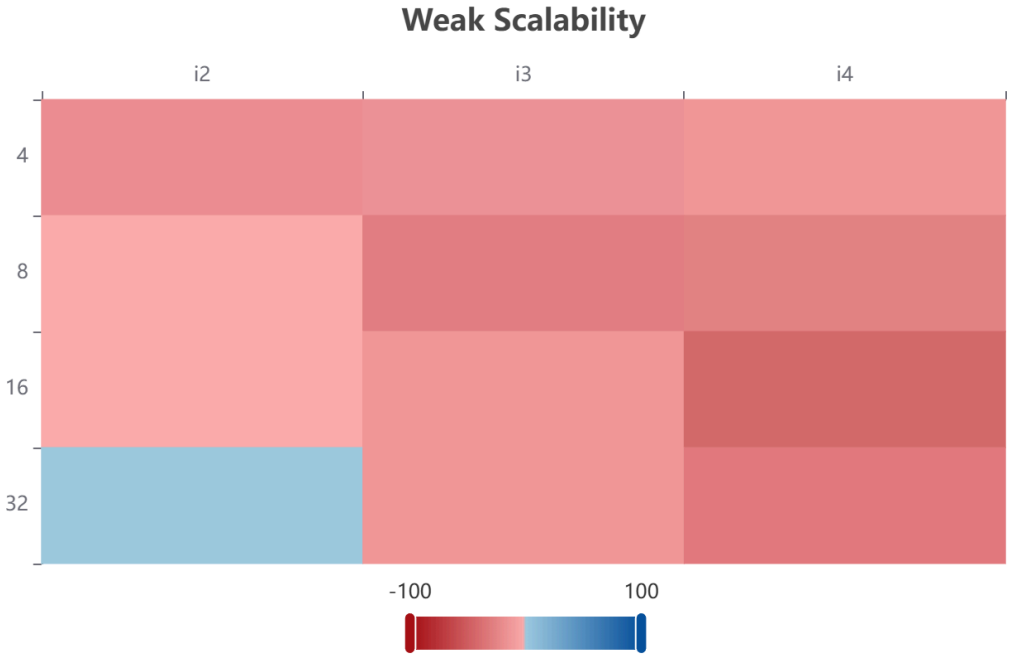


Figura 12: Escalabilidade fraca versão 2

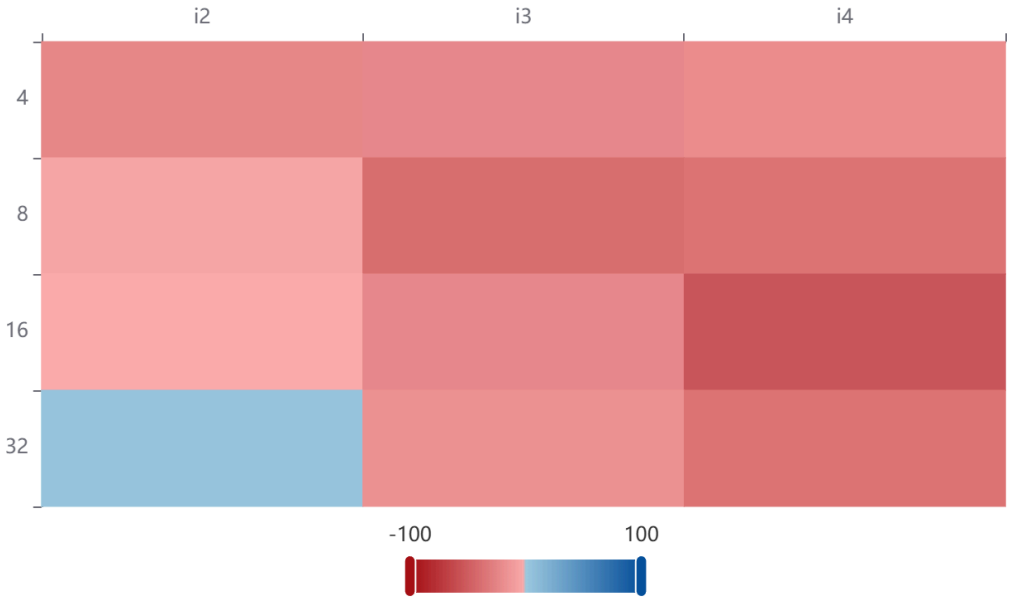
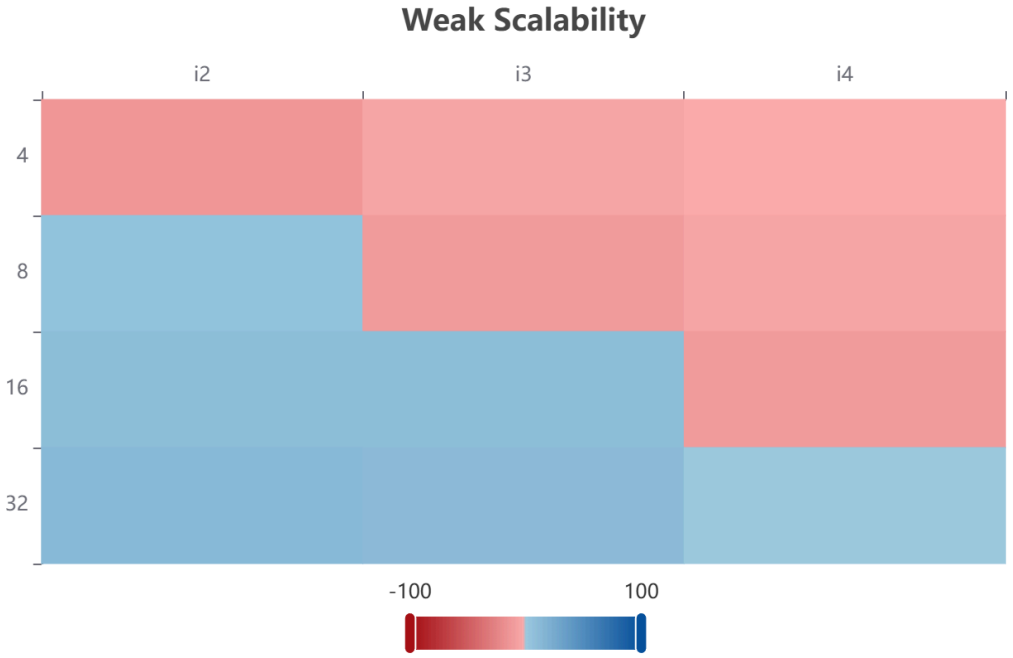


Figura 13: Escalabilidade fraca versão 3



A V3 apresentou o resultado mais favorável ao aumentar conjuntamente o problema e os recursos. Na sequência com carga constante por thread, seu tempo passou de 42,73 para 47,19 ms, enquanto V1 e V2 tiveram aumentos mais expressivos.

A inicialização paralela contribui diretamente para esse resultado: nas primeiras versões, aumentar a malha também aumenta um trabalho executado serialmente. Na V3, essa etapa é distribuída, e a região persistente reduz a sobrecarga da evolução.

5.1. CONSIDERAÇÕES

A comparação mostra que reduzir o tempo de execução e melhorar a escalabilidade são objetivos relacionados, mas distintos. A V2 foi mais rápida que a V1, embora tenha apresentado menor eficiência em relação à própria execução com uma thread. Já a V3 reuniu os melhores resultados paralelos nas maiores entradas: com 32 threads em i4, atingiu 102,48 ms, speedup de 24,14× e eficiência de 75,4%.

A vantagem da V3 resulta da redução de dois gargalos: a inicialização serial, que foi paralelizada, e a sobrecarga das regiões paralelas repetidas, reduzida pela região persistente. Com isso, apresentou melhor escalabilidade geral ao aproveitar

problemas maiores, melhor escalabilidade forte nas maiores malhas e comportamento mais próximo de tempo constante na comparação de escalabilidade fraca com trabalho fixo por thread.

Entretanto, nenhuma versão manteve eficiência ideal em todas as configurações. Nas menores malhas, aumentar as threads deixou de ajudar e, em alguns casos, piorou o tempo. **O experimento evidencia que mais recursos só trazem ganhos quando há trabalho suficiente para compensar a sobrecarga, e que otimizar exige identificar os gargalos do programa completo, avaliando conjuntamente tempo, eficiência e tamanho do problema.**

6. DEMAIS CÓDIGOS

Figura 14: navierStokes.h

```h
#ifndef NAVIER_STOKES_H
#define NAVIER_STOKES_H
#include <stddef.h>

typedef struct { int n; double *u, *v, *un, *vn; } Grade;
void evoluir(Grade *g, int passos);

/* Mesma ordem aritmetica do solver original. CFL: sigma_x+sigma_y=0.4. */
#define ATUALIZAR \
   size_t k=(size_t)i*n+j; \
   un[k]=u[k]+fator*((u[k+n]-2*u[k]+u[k-n])*inv+(u[k+1]-2*u[k]+u[k-1])*inv); \
   vn[k]=v[k]+fator*((v[k+n]-2*v[k]+v[k-n])*inv+(v[k+1]-2*v[k]+v[k-1])*inv)
#endif
```

Figura 15: navierStokes.c

```c
#include <errno.h>
#include <limits.h>
#include <math.h>
#include <omp.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#ifdef USAR_PASCAL
#include "pascalops.h"
```

```
#define INICIAR_REGIAO(id) pascal_start(id)
#define TERMINAR_REGIAO(id) pascal_stop(id)
#else
#define INICIAR_REGIAO(id) ((void)0)
#define TERMINAR_REGIAO(id) ((void)0)
#endif

#include "navierStokes.h"
#ifndef VERSAO
#error Defina VERSAO ao compilar com uma implementacao navierStokesV1, V2 ou V3.
#endif

static int inteiro(const char *s, int minimo, int maximo) {
    char *fim; errno = 0; long x = strtol(s, &fim, 10);
    if (errno || !*s || *fim || x < minimo || x > maximo) {
        fprintf(stderr, "Argumento invalido: %s\n", s); exit(EXIT_FAILURE);
    }
    return (int)x;
}

static Grade criar(int n) {
    Grade g = {.n=n}; size_t bytes = (size_t)n*n*sizeof(double);
    g.u=malloc(bytes); g.v=malloc(bytes); g.un=malloc(bytes); g.vn=malloc(bytes);
    if (!g.u || !g.v || !g.un || !g.vn) { fputs("Falha de alocacao\n",stderr); exit(1); }
    return g;
}

static void liberarMemoria(Grade g) {
    free(g.u);
    free(g.v);
    free(g.un);
    free(g.vn);
}

static void inicializar(Grade g, int paralelo, int cenario) {
    int n=g.n;

    #pragma omp parallel for if(paralelo) schedule(static)
    for (int i=0;i<n;i++) for (int j=0;j<n;j++) {
        size_t k=(size_t)i*n+j;

        double x=(double)i/(n-1)-0.5, y=(double)j/(n-1)-0.5;
        double u=cenario==0 ? 0 : cenario==1 ? 1.5 : 2*exp(-(x*x+y*y)/(2*0.08*0.08));
```

```
      double v=cenario==1 ? 0.5 : u*0.5;

      if (cenario!=1 && (i==0 || j==0 || i==n-1 || j==n-1)) u=v=0;

      g.u[k]=g.un[k]=u; g.v[k]=g.vn[k]=v;
    }
}
static void referencia(Grade *g, int passos) {
    int n=g->n; double inv=(double)(n-1)*(n-1), fator=0.2/inv;
    double *u=g->u,*v=g->v,*un=g->un,*vn=g->vn;
    for (int p=0;p<passos;p++) {
      for (int i=1;i<n-1;i++) for (int j=1;j<n-1;j++) { ATUALIZAR; }
      double *t=u;u=un;un=t;t=v;v=vn;vn=t;
    }
    g->u=u;g->v=v;g->un=un;g->vn=vn;
}

static int validar(void) {
    int falhas=0;
    for (int c=0;c<3;c++) for (int passos=1;passos<=12;passos+=11) {
      Grade ref=criar(31);omp_set_num_threads(1);inicializar(ref,0,c);referencia(&ref,passos);
      for (int th=1;th<=4;th*=2) {
                                                                                Grade
g=criar(31);omp_set_num_threads(th);inicializar(g,VERSAO==3,c);evoluir(&g,passos);
          for (size_t k=0;k<31*31;k++) {
                                  if  (!isfinite(g.u[k])  ||  !isfinite(g.v[k])  ||  fabs(g.u[k]-ref.u[k])>1e-12  ||
fabs(g.v[k]-ref.v[k])>1e-12) falhas++;
              if (c<2 && (g.u[k]!=(c==1?1.5:0) || g.v[k]!=(c==1?0.5:0))) falhas++;
              if (c==2 && (g.u[k]<0 || g.u[k]>2 || fabs(g.v[k]-g.u[k]*0.5)>1e-12)) falhas++;
          }
          liberarMemoria(g);
      }
      liberarMemoria(ref);
    }
      printf("Validacao  V%d:  %s  (%d  falhas)\n",VERSAO,falhas?"FALHOU":"OK",falhas);return
falhas?1:0;
}

int main(int argc,char **argv) {
    omp_set_dynamic(0);
    if (argc==2 && strcmp(argv[1], "--validar")==0) return validar();
#ifdef USAR_PASCAL
    if (argc!=3) {
```

```
                    fprintf(stderr,"Uso: %s PONTOS_INTERNOS PASSOS; threads via
OMP_NUM_THREADS\n",argv[0]);return 1;
  }
  int pontos=inteiro(argv[1],36,268369924);
  int lado=(int)sqrt((double)pontos);
  if ((long long)lado*lado!=pontos) {
    fputs("PONTOS_INTERNOS deve ser um quadrado perfeito\n",stderr);return 1;
  }
  int ver=VERSAO,n=lado+2,passos=inteiro(argv[2],1,1000000),th=omp_get_max_threads();
#else
          if (argc!=4) { fprintf(stderr,"Uso: %s N(8..16384) PASSOS THREADS |
--validar\n",argv[0]);return 1; }
                                                                          int
ver=VERSAO,n=inteiro(argv[1],8,16384),passos=inteiro(argv[2],1,1000000),th=inteiro(argv[3],
1,4096);
#endif
  omp_set_num_threads(th);int reais=0;
  #pragma omp parallel
  {
    #pragma omp single
    reais=omp_get_num_threads();
  }

  if (reais!=th) {
    fprintf(stderr,"Solicitadas %d threads, obtidas %d\n",th,reais);return 1;
  }

  Grade g=criar(n);inicializar(g,ver==3,2);evoluir(&g,5); /* aquecimento */

  INICIAR_REGIAO(1);
  inicializar(g,ver==3,2);
  TERMINAR_REGIAO(1);

  INICIAR_REGIAO(2);
                              double      inicio=omp_get_wtime();evoluir(&g,passos);double
tempo=omp_get_wtime()-inicio;
  TERMINAR_REGIAO(2);

  double soma=0,pico=0;
  for (size_t k=0;k<(size_t)n*n;k++) {
    soma+=g.u[k]+g.v[k];if(g.u[k]>pico)pico=g.u[k];
  }
  if (!isfinite(soma) || pico>2 || pico<0) {
```

```
    fputs("Resultado numerico invalido\n",stderr);
    liberarMemoria(g);
    return 1;
  }

  printf(
    "%d,%d,%d,%d,%.9f,%.17g,%.17g\n",
    ver,
    n,
    passos,
    reais,
    tempo,
    soma,
    pico
  );

  liberarMemoria(g);return 0;
}
```